
(наименование образовательной организации)

ОТЧЁТ

по учебной практике

УП.01 Разработка модулей программного обеспечения для компьютерных систем

УП.02 Осуществление интеграции программных модулей

специальность 09.02.07 Информационные системы и программирование

Тема индивидуального задания: модуль рекомендаций в составе сервиса знакомств «Matchly»

Сроки прохождения практики: 01.09.2026 – 26.09.2026

Место прохождения: учебные лаборатории колледжа

Выполнил: студент группы _____ Высоцкий Александр Алексеевич

_____ (подпись)

Руководитель практики от колледжа: Илюшечкина _____

_____ (подпись)

Профильная организация: Федеральное государственное автономное научное учреждение «Научно-исследовательский институт «Специализированные вычислительные устройства защиты и автоматика», г. Ростов-на-Дону

Руководитель практической подготовки от профильной организации (по согласованию): заместитель директора по научной работе Гуфан К.Ю.

_____ (подпись)

Оценка: _____

Ростов-на-Дону
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ И ВЫБОР ТЕМЫ.....	5
1.1 Требования к разрабатываемому модулю.....	5
1.2 Выбор и обоснование темы.....	6
1.3 Средства разработки.....	6
2 УП.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ.....	8
2.1 Подготовка среды и каркаса проекта.....	8
2.2 Архитектура приложения и применение принципов ООП.....	8
2.3 Модель данных.....	10
2.4 Модуль безопасности и учётных записей.....	10
2.5 Модуль анкет и справочника интересов.....	11
2.6 Модуль реакций и матчей.....	11
2.7 Модуль рекомендаций.....	12
2.8 Модульное и интеграционное тестирование.....	13
3 УП.02 ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ.....	14
3.1 Интеграция модулей в единое приложение.....	14
3.2 Пользовательский интерфейс.....	14
3.3 Единая точка входа и контейнеризация.....	15
3.4 Сквозное тестирование интегрированного приложения.....	16
3.5 Документирование и передача результата.....	17
4 ДНЕВНИК ПРАКТИКИ.....	18
ЗАКЛЮЧЕНИЕ.....	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	21
ПРИЛОЖЕНИЕ А. СТРУКТУРА ПРОЕКТА.....	22
ПРИЛОЖЕНИЕ Б. СХЕМА БАЗЫ ДАННЫХ.....	23
ПРИЛОЖЕНИЕ В. ЭКРАННЫЕ ФОРМЫ.....	25
ПРИЛОЖЕНИЕ Г. ЛИСТИНГИ КЛЮЧЕВЫХ КЛАССОВ.....	32
ПРИЛОЖЕНИЕ Д. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....	38

	Фамилия	Подпись	Дата	Отчёт по учебной практике УП.01, УП.02
Разраб.	Высоцкий А.А.			Лит. Лист Листов Группа _____ Модуль рекомендаций «Matchly»
Провер.	Илюшечкина _____			
Реценз.	Гуфан К.Ю.			
Утв.	Федоров С.Е.			

ВВЕДЕНИЕ

Учебная практика по профессиональным модулям ПМ.01 «Разработка модулей программного обеспечения для компьютерных систем» и ПМ.02 «Осуществление интеграции программных модулей» проходила с 01.09.2026 по 26.09.2026 г. в учебных лабораториях колледжа. Профильная организация, с которой согласована практическая подготовка, – Федеральное государственное автономное научное учреждение «Научно-исследовательский институт «Специализированные вычислительные устройства защиты и автоматика», г. Ростов-на-Дону; руководитель практической подготовки от профильной организации – заместитель директора по научной работе Гуфан К.Ю..

Практика состояла из двух частей. С 01.09 по 12.09.2026 (УП.01) выполнялись проектирование и разработка программных модулей, с 14.09 по 26.09.2026 (УП.02) – их интеграция в единое приложение, разработка пользовательского интерфейса, контейнеризация, сквозное тестирование и документирование.

Цель практики – закрепление теоретических знаний и формирование практического опыта разработки и интеграции программных модулей на промышленном стеке технологий: Java 21, Spring Boot, Spring Security, Spring Data JPA, PostgreSQL, Docker.

Задачи практики:

- проанализировать индивидуальное задание и выбрать тему, соответствующую его требованиям;
- спроектировать архитектуру приложения и модель базы данных;
- реализовать программные модули: безопасность и учётные записи, анкеты и справочник интересов, реакции и матчи, рекомендации;
- обеспечить логирование, единообразную обработку ошибок и документирование кода и API;
- провести модульное и интеграционное тестирование каждого модуля;

- интегрировать модули в единое приложение с REST API и пользовательским интерфейсом;
- обеспечить запуск приложения со всеми зависимостями одной командой на Windows и Linux, в том числе в Docker;
- выполнить сквозное тестирование интегрированного приложения и устранить выявленные дефекты;
- подготовить документацию, разместить проект в системе контроля версий и оформить отчёт.

Объектом разработки стал сервис знакомств «Matchly», ядром которого является модуль персональных рекомендаций (тема 3 индивидуального задания «Модуль рекомендаций»). В отчёте описаны ход и результаты работы по каждому этапу, приведены дневник практики, схема базы данных, экранные формы, листинги ключевых классов и результаты тестирования.

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ И ВЫБОР ТЕМЫ

1.1 Требования к разрабатываемому модулю

Согласно индивидуальному заданию требовалось разработать полноценный backend-модуль на платформе Spring с подключением к базе данных и системой безопасности Spring Security. К проекту предъявлялись следующие требования:

- модульность: разделение кода на логические компоненты (сервисы, модели, интерфейсы, утилиты);
- применение принципов объектно-ориентированного программирования: инкапсуляции, наследования, полиморфизма;
- обязательное логирование и корректная обработка исключений;
- документирование: комментарии, описание модуля, его интерфейсов и классов;
- возможность тестирования модуля отдельно (unit-тесты или демонстрационный класс);
- REST API с чёткой структурой контроллеров, сервисов и репозиторий; базовые CRUD-операции;
- аутентификация и авторизация пользователей (JWT, Basic Auth или сессии);
- использование аннотаций `@RestController`, `@Service`, `@Repository`, `@Entity`; настроенное подключение к базе данных;
- Swagger для документирования API;
- пользовательский интерфейс с комфортной навигацией и UX-ориентированным дизайном.

Результатом должны были стать исходный код модуля, отчёт, демонстрация работы и объяснение способа интеграции модуля в другое приложение.

1.2 Выбор и обоснование темы

Из предложенных тем выбран модуль рекомендаций (Recommendation Module): формирование персональных рекомендаций пользователю, работа с сущностями User, Item, Rating, возможность подключения к любому проекту. Для наглядности модуль реализован в формате сервиса знакомств: пользователь заполняет анкету, оценивает анкеты других людей, а система подбирает подходящих кандидатов и объясняет, почему они показаны. Соответствие сущностей задания и предметной области приведено в таблице 1.

Таблица 1 – Соответствие сущностей задания предметной области

Сущность из задания	Реализация в «Matchly»	Назначение
User	User (учётная запись) и Profile (анкета)	Кто получает рекомендации; настройки поиска (пол, возраст)
Item	Profile другого пользователя	Что рекомендуется: карточка анкеты в ленте
Rating	Reaction (LIKE / SKIP)	Оценка, из которой строится матрица предпочтений

Выбор обоснован тем, что данная предметная область естественно требует нескольких взаимозаменяемых алгоритмов (по интересам, по похожим вкусам, по популярности), то есть наглядно демонстрирует полиморфизм; содержит достаточно бизнес-правил для проверки обработки ошибок и валидации; допускает автономное тестирование алгоритмов на контрольных данных; имеет понятный пользовательский интерфейс.

1.3 Средства разработки

Таблица 2 – Используемые средства разработки

Назначение	Средство	Версия
Язык и платформа	Java (Eclipse Temurin JDK)	21 LTS
Каркас приложения	Spring Boot (Web MVC, Data JPA, Security, Validation, Actuator)	4.1.1
База данных	PostgreSQL	16
Миграции схемы	Flyway	в составе Spring Boot
Документирование API	springdoc-openapi (Swagger UI)	3.1.1

Назначение	Средство	Версия
Сборка	Apache Maven (Maven Wrapper)	3.9
Тестирование	JUnit 5, Mockito, Spring MockMvc, H2 (in-memory)	в составе Spring Boot
Сквозное тестирование интерфейса	Playwright (Node.js), Chromium	1.47
Логирование	SLF4J, Logback	в составе Spring Boot
Интерфейс	HTML, CSS, JavaScript (без сборщика)	—
Контейнеризация	Docker, Docker Compose	29 / 2.39
Контроль версий	Git, GitHub	2.43
Среда разработки	Visual Studio Code, Extension Pack for Java, Spring Boot Extension Pack	—

2 УП.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

2.1 Подготовка среды и каркаса проекта

Каркас проекта сгенерирован средствами Spring Initializr с зависимостями Web MVC, Data JPA, Security, OAuth2 Resource Server, Validation, Flyway, Actuator, драйверами PostgreSQL и H2. В проект включён Maven Wrapper, поэтому для сборки не требуется установленный Maven: нужная версия загружается автоматически, что одинаково работает на Windows и Linux.

Конфигурация вынесена в файл `application.yml`; все значения, зависящие от окружения (адрес и учётные данные базы, порт, секрет подписи токенов, учётная запись администратора), читаются из переменных окружения с разумными значениями по умолчанию. Схема базы данных управляется миграциями Flyway (V1–V4), а Hibernate работает в режиме проверки соответствия сущностей схеме. Логирование настроено через Logback: вывод в консоль и в файл `logs/matchly.log` с ежедневной ротацией; для тестового профиля файл не создаётся.

Для тестов используется встроенная база H2 в режиме совместимости с PostgreSQL, поэтому автотесты не требуют установленного сервера базы данных и Docker.

2.2 Архитектура приложения и применение принципов ООП

Приложение построено по слоистой архитектуре: контроллер принимает HTTP-запрос и валидирует DTO, сервис (интерфейс и реализация) выполняет бизнес-логику в транзакции, репозиторий Spring Data JPA работает с базой, сущность хранит инварианты предметной области. Сущности за пределы сервисного слоя не выходят: контроллеры оперируют неизменяемыми DTO (record), преобразование выполняют мапперы и сборщики. Пакеты проекта соответствуют предметным областям (таблица 3).

Таблица 3 – Пакеты приложения

Пакет	Содержание
config	типобезопасные настройки MatchlyProperties, конфигурация JWT и OpenAPI
common	базовая сущность BaseEntity, иерархия исключений, глобальный обработчик ошибок, утилиты (определение типа изображения, генератор аватаров)
security	цепочка фильтров Spring Security, выдача и разбор JWT, принципал AuthenticatedUser
auth, user, admin	регистрация, вход, удаление аккаунта; учётные записи и роли; административные операции
interest, profile	справочник интересов; анкеты, фото, карточки
reaction, matching	лайки и пропуски; матчи
recommendation	стратегии рекомендаций, контекст, реестр, сервис
settings, bootstrap	настройки, изменяемые администратором; создание администратора и демо-данных при старте

Принципы ООП применены следующим образом.

– Инкапсуляция: сущности не имеют публичных сеттеров; состояние меняется только осмысленными методами (User.block(), Profile.update(...), Reaction.change(...)). Правила совершеннолетия и корректности возрастного диапазона проверяются внутри Profile, поэтому объект невозможно привести в недопустимое состояние.

– Наследование: BaseEntity задаёт идентификатор и метки времени для всех сущностей; MatchlyException – корень иерархии прикладных исключений, каждое из которых знает свой HTTP-статус; AbstractRecommendationStrategy содержит общие методы стратегий.

– Полиморфизм: интерфейс RecommendationStrategy реализуют четыре стратегии, выбираемые во время выполнения через реестр; глобальный обработчик ошибок работает с любым наследником MatchlyException, не зная его конкретного типа.

Обработка ошибок централизована в классе GlobalExceptionHandler (@RestControllerAdvice): любое исключение превращается в ответ формата RFC 9457 application/problem+json с полями status, title, detail, instance, timestamp; ошибки валидации дополняются списком полей. Ошибки безопасности (401, 403) делегируются в тот же обработчик, поэтому формат

ответов единообразен. Каждая обработанная ошибка записывается в лог с уровнем, соответствующим её серьёзности.

2.3 Модель данных

Модель данных состоит из семи таблиц (таблица 4). Все внешние ключи объявлены каскадными: удаление учётной записи удаляет анкету, фото, реакции и матчи одной операцией. SQL миграций написан переносимо и выполняется как в PostgreSQL, так и в H2. Подробное описание столбцов приведено в приложении Б.

Таблица 4 – Таблицы базы данных

Таблица	Назначение	Ключевые связи
users	учётные записи: email, хэш пароля, роль, статус	—
profiles	анкеты: имя, дата рождения, пол, предпочтения, диапазон возраста, город, контакт, видимость	user_id → users (1:1)
interests	справочник интересов с категориями	—
profile_interests	связь анкет и интересов	profile_id, interest_id (M:N)
profile_photos	фото анкеты (bytea), тип содержимого, размер	profile_id → profiles (1:1)
reactions	реакции LIKE / SKIP	from_profile_id, to_profile_id → profiles; пара уникальна
profile_matches	взаимные симпатии	profile_a_id < profile_b_id → profiles; пара уникальна
app_settings	настройки приложения (ключ – значение)	—

2.4 Модуль безопасности и учётных записей

Аутентификация реализована по схеме JWT. При регистрации и входе сервер выдаёт токен, подписанный алгоритмом HMAC-SHA256 средствами Spring Security OAuth2 Resource Server (NimbusJwtEncoder, NimbusJwtDecoder); сторонние библиотеки для работы с JWT не потребовались. Пароли хранятся в виде bcrypt-хэшей. На каждом запросе пользователь читается из базы, поэтому заблокированные и удалённые учётные записи теряют доступ немедленно, а не по истечении срока токена.

Правила доступа: регистрация, вход, Swagger, проверка состояния и статические файлы интерфейса открыты; адреса `/api/admin/**` доступны только роли ADMIN; остальной API требует аутентификации. Администратор создаётся при первом запуске из настроек. Административные операции над учётными записями: поиск по email с пагинацией, блокировка и разблокировка, удаление; администратор не может заблокировать или удалить собственную запись. Пользователь может удалить свой аккаунт, подтвердив пароль.

2.5 Модуль анкет и справочника интересов

Анкета содержит имя, дату рождения, пол, предпочтения по полу и возрасту, город, описание, контакт и от 1 до 10 интересов из общего справочника. Контакт раскрывается только участникам взаимного матча. Правила предметной области (возраст не менее 18 лет, корректный диапазон возраста, количество интересов) проверяются в сущности Profile; формат полей – аннотациями Bean Validation в DTO. Создание и редактирование анкеты выполняются одной идемпотентной операцией PUT `/api/profiles/me`, возвращающей 201 при создании и 200 при обновлении.

Фото хранится в базе данных как bytea в отдельной сущности, чтобы тяжёлые данные не загружались вместе с анкетой при построении рекомендаций. Допускаются JPEG, PNG и WebP до 2 МБ; тип определяется по сигнатуре файла, а не по заголовку от клиента. Ссылка на фото содержит версию, что позволяет браузеру кэшировать изображение и обновлять его после замены. Справочник интересов (35 стартовых элементов в шести категориях) ведёт администратор; удаление интереса каскадно убирает его из анкет.

2.6 Модуль реакций и матчей

Реакция – это оценка одной анкеты другой: LIKE или SKIP. Пара анкет в таблице реакций уникальна, повторная реакция меняет тип, а не создаёт дубликат. Взаимный лайк создаёт матч; пара в матче хранится

упорядоченной по идентификатору, поэтому матч А–В и В–А – одна запись. Пропуск после матча разрывает его; разрыв матча кнопкой переводит собственный лайк в пропуск, чтобы анкета не вернулась в ленту. Нельзя реагировать на собственную, скрытую или принадлежащую заблокированному пользователю анкету.

2.7 Модуль рекомендаций

Модуль реализует конвейер: жёсткие фильтры кандидатов (анкета видима, владелец не заблокирован, это не сам пользователь, на анкету ещё не было реакции, предпочтения по полу и возрасту взаимны) – построение контекста (текущая дата, матрица лайков «кто кого лайкнул», число полученных лайков; загружается один раз на запрос) – оценка кандидатов выбранной стратегией числом от 0 до 1 с перечнем причин – сортировка и выдача первых N карточек.

Реализованы четыре стратегии (паттерн «Стратегия»).

– По интересам (CONTENT): $\text{score} = 0,6 \cdot J + 0,25 \cdot A + 0,15 \cdot C$, где J – коэффициент Жаккара множеств интересов, $A = \max(0; 1 - |\text{разница возрастов}| / 15)$, $C = 1$ при совпадении города. Работает для новых пользователей без лайков.

– Похожие вкусы (COLLABORATIVE): коллаборативная фильтрация по пользователям. Похожесть $\text{sim}(v, u) = |L(v) \cap L(u)| / \sqrt{(|L(v)| \cdot |L(u)|)}$ (косинусная близость множеств лайков); оценка кандидата – взвешенная доля похожих пользователей, лайкнувших его. Без собственных лайков стратегия возвращает нули (холодный старт).

– Популярные (POPULARITY): $\text{score} = \text{лайки}(c) / \text{максимум лайков среди кандидатов}$.

– Умная подборка (HYBRID): $0,5 \cdot \text{content} + 0,3 \cdot \text{collaborative} + 0,2 \cdot \text{popularity} + 0,15 \cdot [\text{кандидат уже лайкнул пользователя}]$, результат ограничивается единицей; причины всех составляющих объединяются.

Реестр StrategyRegistry собирает все реализации из контекста Spring и при старте проверяет, что для каждого значения перечисления StrategyType есть стратегия; добавление нового алгоритма сводится к одному классу. Алгоритм по умолчанию хранится в таблице настроек и изменяется администратором без перезапуска; пользователь может выбрать любой алгоритм в интерфейсе. Каждая карточка сопровождается человекочитаемыми причинами («Общие интересы: Кино, Кофе», «Нравится 4 пользователям с похожими вкусами», «Проявил(а) интерес к вам»).

2.8 Модульное и интеграционное тестирование

Каждый модуль тестировался сразу после реализации, до перехода к следующему. Unit-тесты (JUnit 5, Mockito) проверяют сервис аутентификации с замканными зависимостями, выдачу и проверку JWT, доменные правила анкеты, совместимость предпочтений, логику матча, определение типа изображения и все четыре стратегии на небольших матрицах с заранее рассчитанными значениями (идентичные анкеты дают 1,0, непохожие – 0, косинусная близость взвешивает «советчиков», холодный старт даёт нули, оценка гибрида не превышает единицу).

Интеграционные API-тесты (полный контекст Spring, MockMvc, H2) проверяют регистрацию и вход, формат ошибок, права доступа, мгновенный отзыв доступа при блокировке, каскадное удаление, анкеты и фото, справочник, реакции и матчи, рекомендации и настройки, статистику, генератор демо-данных, а также граничные значения: возраст 18 и 99 лет, ровно 10 интересов, пустой и слишком большой файл, лимиты пагинации, нечисловые параметры, неизвестные адреса и методы. Итог первой части практики – 89 автотестов, все проходят; сводка приведена в приложении Д. Дополнительно каждый модуль проверялся вживую на PostgreSQL 16: старт приложения, применение миграций, запросы через curl, содержимое таблиц.

3 УП.02 ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ

3.1 Интеграция модулей в единое приложение

Разработанные модули объединены в одно Spring Boot-приложение с общим REST API (таблица 5), единым форматом ошибок, единой конфигурацией безопасности и общей документацией OpenAPI: каждая операция снабжена аннотациями @Tag, @Operation, @ApiResponse, а Swagger UI поддерживает авторизацию токеном. Для наполнения интегрированного приложения при первом запуске генерируются демо-данные: 60 анкет с SVG-аватарами, интересами, реакциями и матчами; генерация детерминирована и идиempotentна.

Таблица 5 – REST API интегрированного приложения

Метод и путь	Назначение	Доступ
POST /api/auth/register, POST /api/auth/login	регистрация и вход, выдача JWT	все
GET, DELETE /api/auth/me	текущий аккаунт; удаление своего аккаунта	пользователь
GET /api/interests	справочник интересов	пользователь
GET, PUT /api/profiles/me	своя анкета: создание и редактирование	пользователь
PUT, DELETE /api/profiles/me/photo	загрузка и удаление фото	пользователь
GET /api/profiles/{id}, GET /api/profiles/{id}/photo	карточка и фото другой анкеты	пользователь
GET /api/recommendations, GET /api/recommendations/strategies	подборка с объяснениями; список алгоритмов	пользователь
POST /api/reactions	лайк или пропуск; при взаимности возвращает матч	пользователь
GET /api/matches, DELETE /api/matches/{id}	матчи с контактами; разрыв матча	пользователь
GET /api/admin/users..., POST .../block, .../unblock, DELETE ...	учётные записи: поиск, блокировка, удаление	администратор
POST, PUT, DELETE /api/admin/interests	ведение справочника	администратор
GET, PUT /api/admin/settings	алгоритм по умолчанию	администратор
GET /api/admin/stats	сводная статистика	администратор

3.2 Пользовательский интерфейс

Интерфейс выполнен как одностраничное приложение на HTML, CSS и JavaScript без сборщика и фреймворков; оно обслуживается тем же Spring

Boot из каталога static и работает с REST API через JWT, то есть одновременно демонстрирует API. Экраны: вход и регистрация; онбординг и редактирование анкеты с выбором интересов по категориям и загрузкой фото; лента «Знакомства» с карточкой, оценкой совпадения, причинами показа, переключателем алгоритма и горячими клавишами; матчи с контактами и кнопкой копирования; панель администратора со статистикой, пользователями, справочником и настройками. Экранные формы приведены в приложении В.

UX-решения: онбординг открывается автоматически, пока анкета не заполнена; взаимный лайк показывает окно «Это матч!» с контактом; пустые состояния подсказывают следующее действие; опасные действия требуют подтверждения; ошибки сервера показываются под соответствующими полями формы или уведомлением, причём сообщения API переводятся на русский язык словарём на стороне клиента; выбранный алгоритм запоминается; фото загружаются защищёнными запросами и отображаются через blob-URL; вёрстка адаптивна вплоть до ширины 400 px.

3.3 Единая точка входа и контейнеризация

Для запуска приложения со всеми зависимостями одной командой созданы скрипты start.sh (Linux, macOS) и start.cmd (Windows) с одинаковой логикой: проверка наличия Java 21, проверка доступности PostgreSQL, автоматический запуск базы в Docker при её отсутствии, запуск приложения вместе с интерфейсом. Режим --docker собирает образ и поднимает базу и приложение целиком в Docker (Java на машине не требуется), режимы --stop и --test останавливают контейнеры и запускают автотесты. Настройки читаются из файла .env.

Образ приложения описан многоэтапным Dockerfile: сборка jar выполняется внутри контейнера с JDK, в итоговый образ попадают только JRE и jar, приложение работает от непривилегированного пользователя. Файл docker-compose.yml описывает службы db (PostgreSQL 16 с проверкой

готовности) и app (зависит от готовности базы, имеет проверку состояния, хранит данные и логи в томах). Кроссплатформенность обеспечена относительными путями, кодировкой UTF-8, ASCII-скриптом для Windows и тестами на встроенной H2, не требующими внешних служб.

3.4 Сквозное тестирование интегрированного приложения

Интегрированное приложение проверялось в реальном браузере Chromium средствами Playwright. Основной сценарий из 18 шагов проходит путь пользователя от регистрации до панели администратора: онбординг с загрузкой фото, лента с объяснением, взаимный лайк и окно матча, список матчей, пропуск клавиатурой, редактирование анкеты, разрыв матча, вход администратора, статистика, поиск, блокировка и разблокировка пользователя, добавление и удаление интереса, смена алгоритма по умолчанию, мобильная ширина.

Сценарий корнер-кейсов из 19 групп проверяет маршруты без сессии, испорченный токен, переводы ошибок, глубокие ссылки без анкеты, подсветку ошибок под полями, лимит интересов, провал загрузки фото без потери анкеты, пустую ленту, разметку в имени и описании (защита от XSS), сохранение выбранного алгоритма, двойной клик по кнопке, горячие клавиши при открытом окне, блокировку во время работы, удаление аккаунта, поиск и пагинацию в панели администратора, отсутствие горизонтальной прокрутки на 400 px для всех экранов. Оба сценария выполнены как против локального запуска, так и против стека в Docker.

Сквозные тесты выявили дефекты, устранённые в ходе практики: пустая лента у части демо-пользователей (генератор данных оценивал за них всех подходящих кандидатов); локализация сообщений валидации по заголовку браузера (локаль API зафиксирована, перевод выполняет интерфейс); выход из аккаунта при неверном пароле подтверждения удаления (сервер отвечал 401, теперь 422). Вживую проверен ответ 413 на

файл больше 2 МБ и сохранность данных в Docker при перезапуске контейнеров.

3.5 Документирование и передача результата

Подготовлена документация: README с инструкциями по установке и запуску на Windows и Linux (три варианта установки базы, единая точка входа, переменные окружения, типичные проблемы); описание архитектуры и алгоритмов; инструкция по интеграции модуля рекомендаций в другое приложение тремя способами – как отдельный сервис по REST, как Java-библиотека с адаптером под свои сущности и как расширение новой стратегией; скриншоты экранов; описание браузерных тестов. Код снабжён Javadoc, миграции и конфигурация – комментариями.

Проект размещён в системе контроля версий Git и опубликован в приватном репозитории GitHub (github.com/YaroslavalSORA/matchly) с последующей передачей прав. Итоговый объём: 111 файлов исходного кода Java основного модуля, 22 файла тестов, 4 миграции базы данных, интерфейс из четырёх файлов, два браузерных сценария.

4 ДНЕВНИК ПРАКТИКИ

УП.01 Разработка модулей программного обеспечения для компьютерных систем (01.09–12.09.2026)

Таблица 6 – Дневник практики, модуль УП.01

Дата	Содержание выполненной работы	Отметка руководителя
01.09.2026	Организационное собрание, инструктаж по охране труда и технике безопасности. Получение и анализ индивидуального задания, изучение требований к модулю.	
02.09.2026	Выбор темы (модуль рекомендаций в формате сервиса знакомств) и её обоснование. Подготовка среды: JDK 21, PostgreSQL 16, Maven Wrapper; генерация каркаса Spring Boot.	
03.09.2026	Проектирование архитектуры: слои, пакеты, соглашения. Настройка конфигурации, логирования Logback и миграций Flyway; миграция V1 (учётные записи).	
04.09.2026	Модуль безопасности: выдача и проверка JWT, регистрация, вход, роли; единый обработчик ошибок в формате ProblemDetail.	
05.09.2026	Административные операции над учётными записями: поиск, блокировка, удаление. Unit- и API-тесты модуля безопасности.	
07.09.2026	Модель анкеты и справочника интересов: миграция V2, сущности, доменные правила, DTO и мапперы.	
08.09.2026	Загрузка и выдача фото с хранением в базе, определение типа по сигнатуре файла. Тесты модуля анкет и справочника.	
09.09.2026	Модуль реакций и матчей: миграция V3, сущности, сервисы, REST API, тесты.	
10.09.2026	Модуль рекомендаций: контекст, стратегии по интересам и по популярности, реестр стратегий, сервис-конвейер.	
11.09.2026	Коллаборативная и гибридная стратегии, настройка алгоритма по умолчанию (миграция V4). Unit-тесты алгоритмов на контрольных матрицах.	
12.09.2026	Генератор демо-данных, проверка всех модулей на PostgreSQL, устранение дефекта пустой ленты. Подведение итогов первой части практики.	

УП.02 Осуществление интеграции программных модулей (14.09–26.09.2026)

Таблица 7 – Дневник практики, модуль УП.02

Дата	Содержание выполненной работы	Отметка руководителя
14.09.2026	Анализ требований к интеграции модулей: единое приложение, интерфейс, единая точка входа. Планирование работ второй части практики.	
15.09.2026	Интеграция модулей через общий REST API: статистика для администратора, удаление аккаунта, каскадные связи, описания OpenAPI.	
16.09.2026	Пользовательский интерфейс: каркас одностраничного приложения, клиент API, экраны входа и регистрации,	

Дата	Содержание выполненной работы	Отметка руководителя
	онбординг анкеты.	
17.09.2026	Экраны ленты рекомендаций, матчей и редактирования анкеты; работа с фото через защищённые запросы.	
18.09.2026	Панель администратора: статистика, пользователи, интересы, настройки. Адаптивная вёрстка.	
19.09.2026	Сквозной браузерный сценарий Playwright (18 шагов); исправление выявленных дефектов интерфейса.	
21.09.2026	Контейнеризация: Dockerfile, docker-compose.yml, файл .env; единая точка входа start.sh и start.cmd.	
22.09.2026	Проверка запуска в Docker, сохранности данных при перезапуске, работы скриптов; учёт требований к запуску на Windows и Linux.	
23.09.2026	Тесты корнер-кейсов интерфейса (19 групп) и граничные тесты API; устранение дефектов (локаль сообщений, удаление аккаунта).	
24.09.2026	Документирование: README для Windows и Linux, описание архитектуры, инструкция по интеграции модуля в другие приложения.	
25.09.2026	Размещение проекта в системе контроля версий (GitHub), подготовка демонстрации работы.	
26.09.2026	Оформление отчёта по практике, защита результатов.	

ЗАКЛЮЧЕНИЕ

В ходе учебной практики выполнены все задачи индивидуального задания. В первой части (УП.01) спроектированы и реализованы программные модули сервиса знакомств «Matchly»: безопасность и учётные записи на основе Spring Security и JWT, анкеты со справочником интересов и фото, реакции и матчи, модуль рекомендаций с четырьмя взаимозаменяемыми стратегиями и объяснением результатов. Каждый модуль имеет чёткую слоистую структуру, документирован и покрыт тестами.

Во второй части (УП.02) модули интегрированы в единое приложение с REST API, документацией Swagger и пользовательским интерфейсом; обеспечен запуск приложения со всеми зависимостями одной командой на Windows и Linux, в том числе полностью в Docker; выполнено сквозное тестирование в реальном браузере, выявленные дефекты устранены; подготовлена документация, проект размещён в системе контроля версий.

Количественные результаты: 89 автотестов (36 модульных и 53 интеграционных), два браузерных сценария (18 шагов и 19 групп проверок), 111 файлов исходного кода Java, 4 миграции базы данных, 60 демонстрационных анкет. Все проверки успешно пройдены как при локальном запуске, так и в контейнерах.

В результате практики закреплены навыки проектирования слоистой архитектуры, применения принципов ООП, работы с Spring Boot, Spring Security, Spring Data JPA и PostgreSQL, написания модульных, интеграционных и сквозных тестов, контейнеризации и документирования. Возможные направления развития проекта: обмен сообщениями внутри матча, несколько фото в анкете, учёт расстояния между городами, обучение весов гибридной стратегии на реальных матчах, кэширование матрицы лайков для больших объёмов данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation [Электронный ресурс]. – URL: <https://docs.spring.io/spring-boot/> (дата обращения: 20.09.2026).
2. Spring Security Reference [Электронный ресурс]. – URL: <https://docs.spring.io/spring-security/reference/> (дата обращения: 20.09.2026).
3. Spring Data JPA Reference Documentation [Электронный ресурс]. – URL: <https://docs.spring.io/spring-data/jpa/reference/> (дата обращения: 20.09.2026).
4. PostgreSQL 16 Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 20.09.2026).
5. Flyway Documentation [Электронный ресурс]. – URL: <https://documentation.red-gate.com/flyway> (дата обращения: 20.09.2026).
6. RFC 7519. JSON Web Token (JWT) [Электронный ресурс]. – URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата обращения: 20.09.2026).
7. RFC 9457. Problem Details for HTTP APIs [Электронный ресурс]. – URL: <https://www.rfc-editor.org/rfc/rfc9457> (дата обращения: 20.09.2026).
8. OpenAPI Specification v3.1 [Электронный ресурс]. – URL: <https://spec.openapis.org/oas/latest.html> (дата обращения: 20.09.2026).
9. springdoc-openapi Documentation [Электронный ресурс]. – URL: <https://springdoc.org/> (дата обращения: 20.09.2026).
10. JUnit 5 User Guide [Электронный ресурс]. – URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 20.09.2026).
11. Playwright Documentation [Электронный ресурс]. – URL: <https://playwright.dev/docs/intro> (дата обращения: 20.09.2026).
12. Docker Documentation [Электронный ресурс]. – URL: <https://docs.docker.com/> (дата обращения: 20.09.2026).
13. Блох, Дж. Java. Эффективное программирование / Дж. Блох. – 3-е изд. – М.: Диалектика, 2019. – 464 с.
14. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – СПб.: Питер, 2018. – 352 с.
15. Уоллс, К. Spring в действии / К. Уоллс. – 6-е изд. – М.: ДМК Пресс, 2022. – 544 с.
16. Аггарвал, Ч. Рекомендательные системы. Учебник / Ч. Аггарвал. – М.: ДМК Пресс, 2021. – 542 с.

ПРИЛОЖЕНИЕ А. СТРУКТУРА ПРОЕКТА

matchly/	
├─ start.sh, start.cmd	единая точка входа (Linux/macOS и Windows)
├─ Dockerfile, docker-compose.yml	образ приложения и запуск всего стека в Docker
├─ .env.example	шаблон настроек
├─ pom.xml, mvnw, mvnw.cmd	сборка Maven (Maven Wrapper)
├─ infra/	init-db.sql, pg-local.sh
├─ e2e/	браузерные тесты Playwright (e2e.js, corner.js,
shots.js)	
├─ docs/	REPORT.md, INTEGRATION.md, screenshots/
├─ src/	
├─ main/java/com/matchly/	
├─ MatchlyApplication.java	точка входа
├─ config/	MatchlyProperties, JwtConfig, OpenApiConfig
├─ common/	BaseEntity, исключения, GlobalExceptionHandler,
утилиты	
├─ security/	SecurityConfig, JwtTokenService,
JwtUserAuthenticationConverter	
├─ auth/	AuthController, AuthService, DTO
├─ user/	User, Role, UserStatus, UserRepository,
UserService	
├─ admin/	контроллеры администратора, статистика
├─ interest/	Interest, InterestService, InterestController
├─ profile/	Profile, ProfilePhoto, ProfileService,
ProfileController	
├─ reaction/	Reaction, ReactionService, ReactionController
├─ matching/	Match, MatchService, MatchController
├─ recommendation/	RecommendationStrategy, strategy/*,
StrategyRegistry, сервис	
├─ settings/	AppSetting, SettingsService
├─ bootstrap/	AdminInitializer, DemoDataSeeder
main/resources/	
├─ application.yml, logback-spring.xml	
├─ db/migration/V1..V4	миграции Flyway
├─ static/	index.html, css/app.css, js/app.js, js/api.js
test/java/com/matchly/	unit- и API-тесты (22 файла)

ПРИЛОЖЕНИЕ Б. СХЕМА БАЗЫ ДАННЫХ

Таблица Б.1 – Таблица users

Столбец	Тип и ограничения
id	BIGINT, PK, identity
email	VARCHAR(255), уникальный
password_hash	VARCHAR(100)
role	VARCHAR(20): USER ADMIN
status	VARCHAR(20): ACTIVE BLOCKED
created_at, updated_at	TIMESTAMP WITH TIME ZONE

Таблица Б.2 – Таблица profiles

Столбец	Тип и ограничения
id	BIGINT, PK
user_id	BIGINT, FK → users, уникальный, ON DELETE CASCADE
display_name	VARCHAR(50)
birth_date	DATE
gender	VARCHAR(10): MALE FEMALE OTHER
looking_for	VARCHAR(10): MALE FEMALE EVERYONE
age_min, age_max	INT
city	VARCHAR(100), индекс
bio	VARCHAR(1000)
contact	VARCHAR(100)
visible	BOOLEAN
created_at, updated_at	TIMESTAMP WITH TIME ZONE

Таблица Б.3 – Таблица interests

Столбец	Тип и ограничения
id	BIGINT, PK
name	VARCHAR(50), уникальный
category	VARCHAR(30): ACTIVE CULTURE FOOD HOBBY LIFESTYLE TECH

Таблица Б.4 – Таблица profile_interests

Столбец	Тип и ограничения
profile_id	BIGINT, FK → profiles, ON DELETE CASCADE
interest_id	BIGINT, FK → interests, ON DELETE CASCADE
(profile_id, interest_id)	первичный ключ

Таблица Б.5 – Таблица profile_photos

Столбец	Тип и ограничения
id	BIGINT, PK
profile_id	BIGINT, FK → profiles, уникальный, ON DELETE CASCADE
content_type	VARCHAR(50)
size_bytes	INT
data	BYTEA

Столбец	Тип и ограничения
created_at, updated_at	TIMESTAMP WITH TIME ZONE

Таблица Б.6 – Таблица reactions

Столбец	Тип и ограничения
id	BIGINT, PK
from_profile_id	BIGINT, FK → profiles, ON DELETE CASCADE
to_profile_id	BIGINT, FK → profiles, ON DELETE CASCADE
reaction_type	VARCHAR(10): LIKE SKIP
(from_profile_id, to_profile_id)	уникальная пара; индекс по (to_profile_id, reaction_type)
created_at, updated_at	TIMESTAMP WITH TIME ZONE

Таблица Б.7 – Таблица profile_matches

Столбец	Тип и ограничения
id	BIGINT, PK
profile_a_id	BIGINT, FK → profiles, ON DELETE CASCADE
profile_b_id	BIGINT, FK → profiles, ON DELETE CASCADE
(profile_a_id, profile_b_id)	уникальная пара, profile_a_id < profile_b_id
created_at, updated_at	TIMESTAMP WITH TIME ZONE

Таблица Б.8 – Таблица app_settings

Столбец	Тип и ограничения
setting_key	VARCHAR(50), PK
setting_value	VARCHAR(255)
updated_at	TIMESTAMP WITH TIME ZONE

ПРИЛОЖЕНИЕ В. ЭКРАННЫЕ ФОРМЫ

♥ Matchly

Войти Регистрация

Знакомства, которые подбирают за вас

Matchly сравнивает интересы, учится на лайках и показывает тех, кто действительно подходит. И честно объясняет, почему.

Вход

Email
you@example.com

Пароль

Войти

Нет аккаунта? [Зарегистрируйтесь](#)

Демо-доступ: demo1@matchly.local / demo1234

Рисунок В.1 – Экран входа

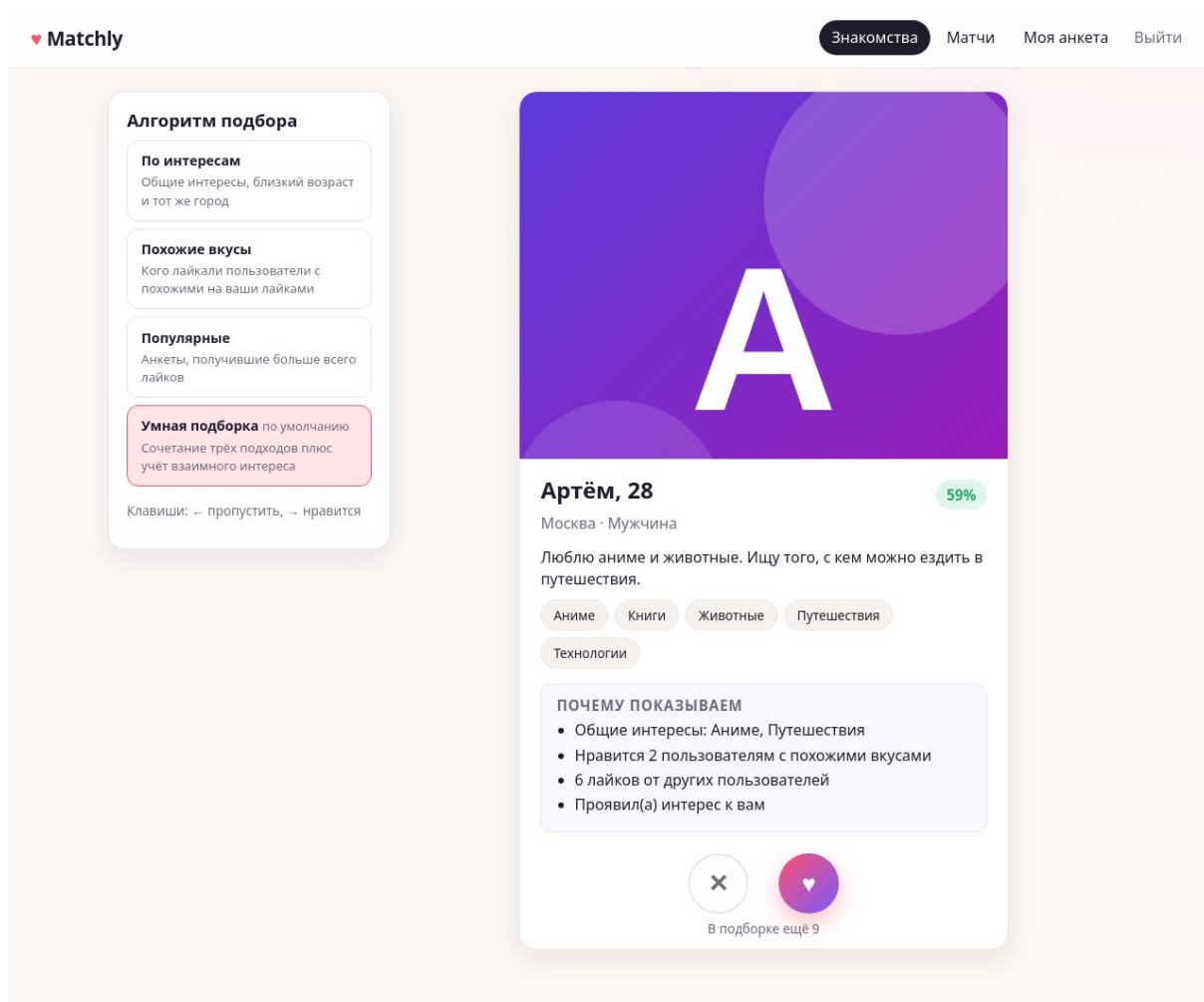


Рисунок В.2 – Лента «Знакомства»: карточка, оценка совпадения, причины показа, выбор алгоритма

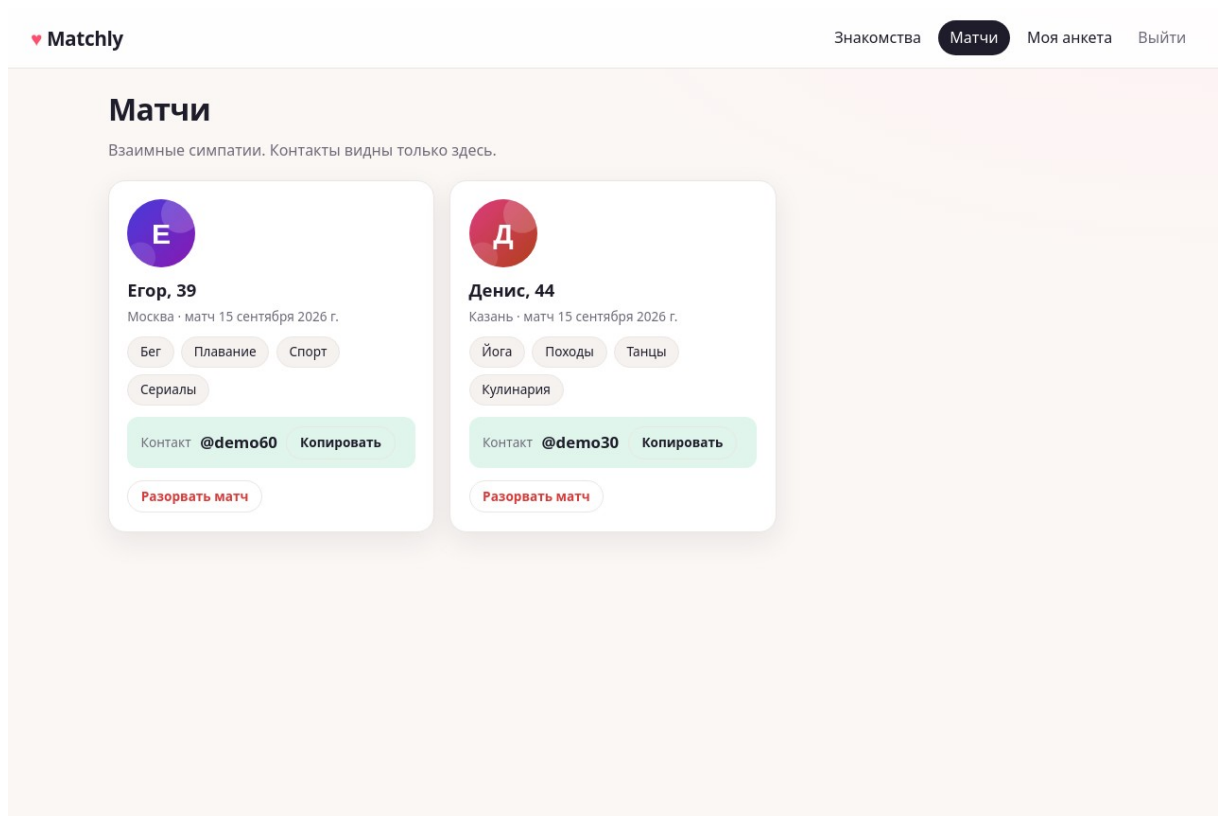


Рисунок В.3 – Матчи с контактами

Моя анкета

Изменения сразу влияют на подборку.

[Выбрать фото](#)[Удалить фото](#)

JPEG, PNG или WebP до 2 МБ

Имя**Дата рождения****Пол****Кого ищу****Возраст от****Возраст до****Город****Контакт для матчей**

Увидят только те, с кем случится взаимный лайк

О себе**Интересы** 3/10**АКТИВНОСТЬ****КУЛЬТУРА****ЕДА И НАПИТКИ****ХОББИ****ОБРАЗ ЖИЗНИ****ТЕХНОЛОГИИ**

Рисунок В.4 – Редактирование анкеты

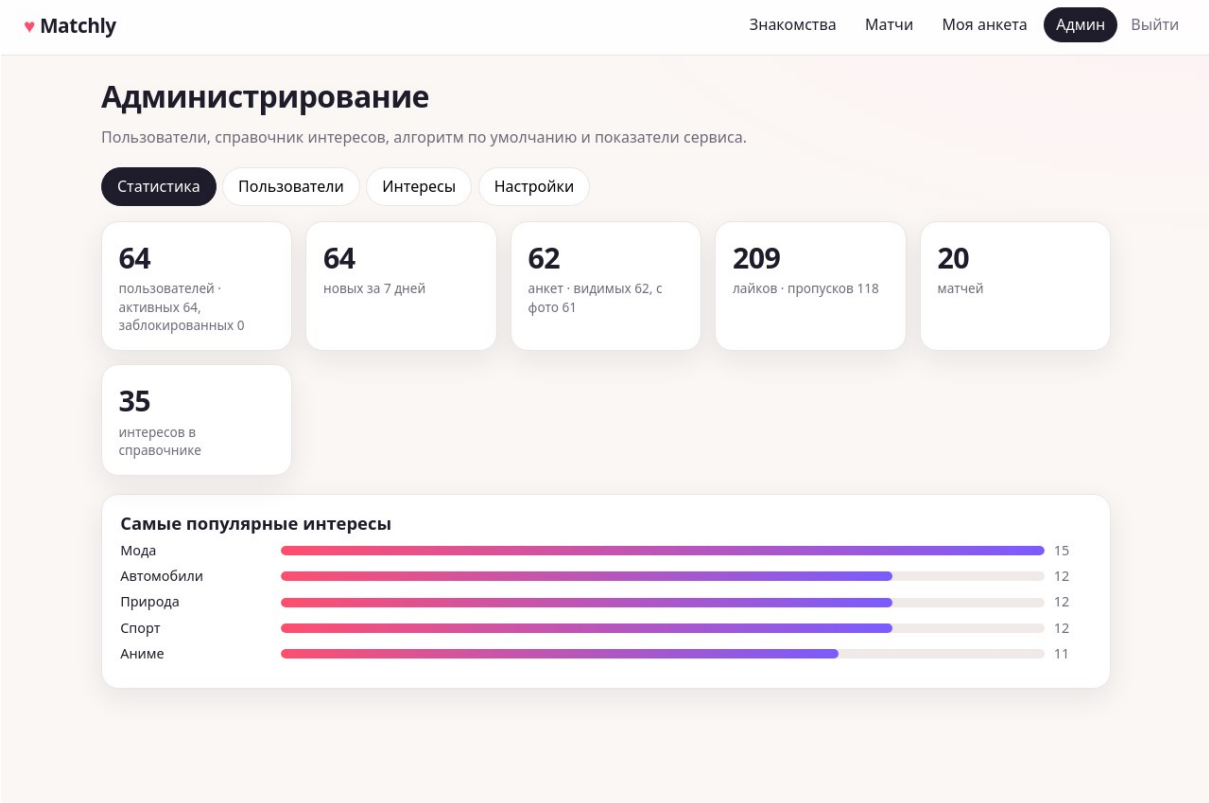


Рисунок В.5 – Панель администратора: статистика

♥ Matchly

ЗнакомстваМатчиМоя анкетаАдминВыйти

Администрирование

Пользователи, справочник интересов, алгоритм по умолчанию и показатели сервиса.

Статистика

Пользователи

Интересы

Настройки

Поиск по email

Всего: 64

ID	EMAIL	РОЛЬ	СТАТУС	СОЗДАН	ДЕЙСТВИЯ	
124	demo60@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
123	demo59@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
122	demo58@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
121	demo57@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
120	demo56@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
119	demo55@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
118	demo54@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
117	demo53@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
116	demo52@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
115	demo51@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить
114	demo50@matchly.local	USER	Активен	15 сентября 2026 г.	Заблокировать	Удалить

Рисунок В.6 – Панель администратора: пользователи

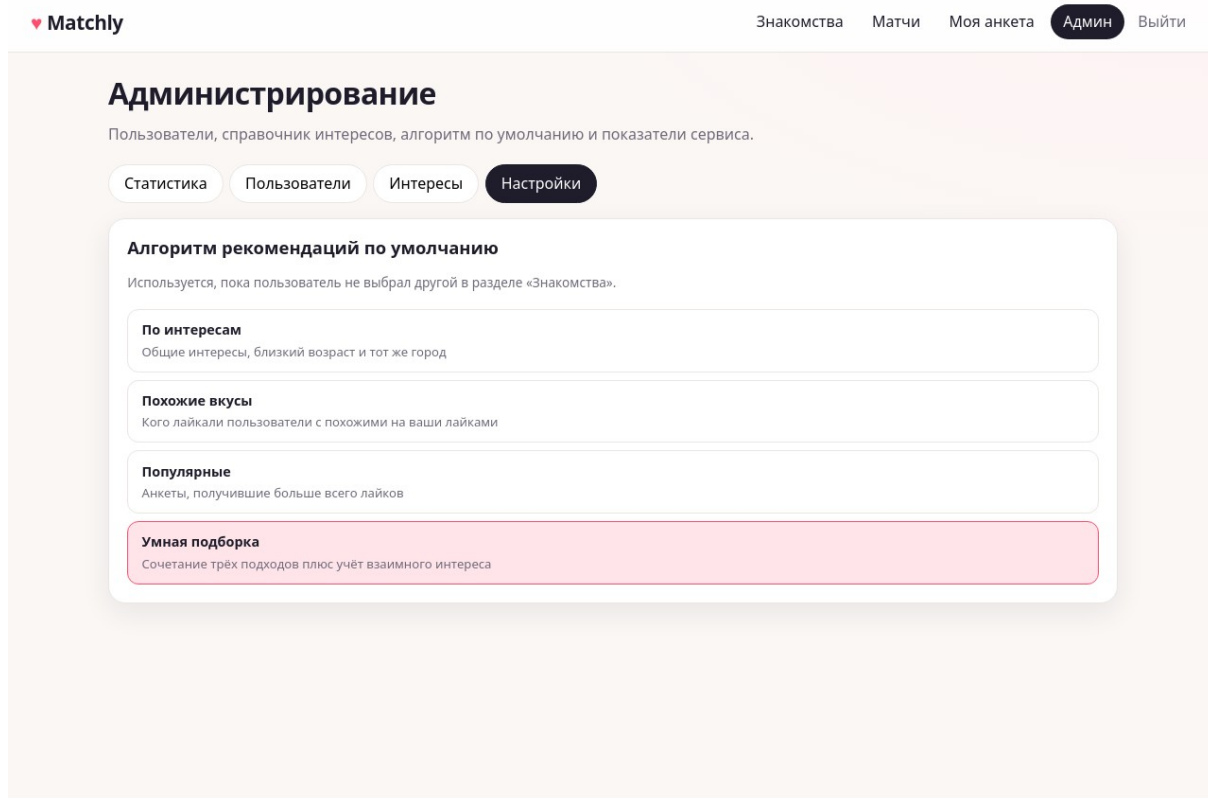


Рисунок В.7 – Панель администратора: алгоритм по умолчанию

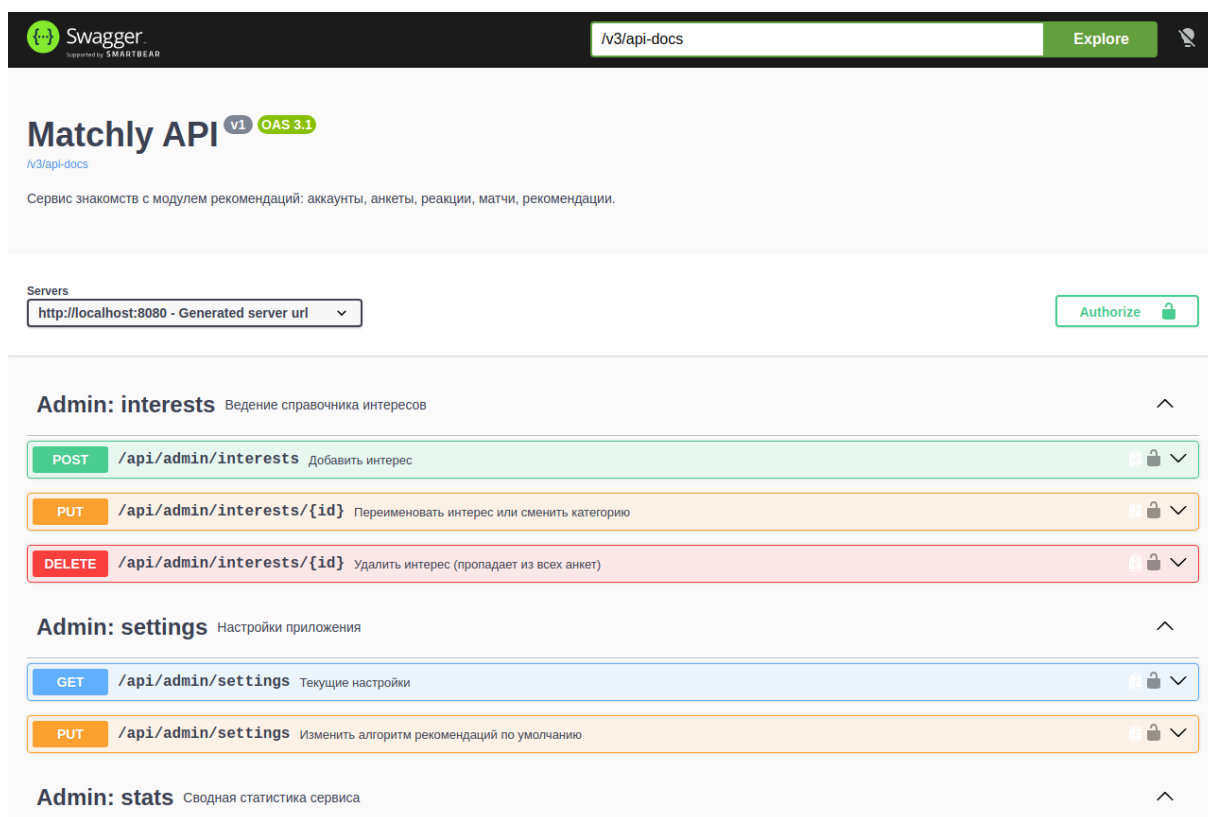


Рисунок В.8 – Документация API в Swagger UI



Рисунок В.9 – Мобильная вёрстка ленты (ширина 400 px)

ПРИЛОЖЕНИЕ Г. ЛИСТИНГИ КЛЮЧЕВЫХ КЛАССОВ

Листинг Г.1 – Интерфейс стратегии рекомендаций RecommendationStrategy

```
package com.matchly.recommendation;

import com.matchly.profile.Profile;
import java.util.List;

/**
 * Стратегия ранжирования кандидатов (паттерн «Стратегия»).
 * Реализации взаимозаменяемы: сервис выбирает нужную по {@link StrategyType} во время
 * выполнения.
 */
public interface RecommendationStrategy {

    StrategyType type();

    /**
     * Оценивает кандидатов для зрителя. Порядок результата не важен, сервис сортирует
     * сам.
     * Каждая оценка лежит в диапазоне 0..1, чтобы стратегии можно было комбинировать.
     */
    List<ScoredCandidate> rank(Profile viewer, List<Profile> candidates,
        RecommendationContext context);
}
```

Листинг Г.2 – Стратегия по интересам ContentBasedStrategy

```
package com.matchly.recommendation.strategy;

import com.matchly.interest.Interest;
import com.matchly.profile.Profile;
import com.matchly.recommendation.RecommendationContext;
import com.matchly.recommendation.ScoredCandidate;
import com.matchly.recommendation.StrategyType;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/**
 * Рекомендации по содержимому анкет: сходство интересов (коэффициент Жаккара),
 * близость возраста и совпадение города. Работает даже для новых пользователей без
 * лайков.
 */
@Component
public class ContentBasedStrategy extends AbstractRecommendationStrategy {

    static final double WEIGHT_INTERESTS = 0.6;
    static final double WEIGHT_AGE = 0.25;
    static final double WEIGHT_CITY = 0.15;
    /** Разница в возрасте, при которой возрастная составляющая обнуляется. */
    static final int AGE_SPAN_YEARS = 15;
    static final int CLOSE_AGE_YEARS = 3;
    private static final int MAX_LISTED_INTERESTS = 3;

    @Override
    public StrategyType type() {
        return StrategyType.CONTENT;
    }

    @Override
    public List<ScoredCandidate> rank(Profile viewer, List<Profile> candidates,
        RecommendationContext context) {
```



```

        return candidates.stream().map(candidate -> score(viewer, candidate,
context)).toList());
    }

    ScoredCandidate score(Profile viewer, Profile candidate, RecommendationContext
context) {
        List<Interest> common = viewer.commonInterests(candidate);
        int union = viewer.getInterests().size() + candidate.getInterests().size() -
common.size();
        double interestScore = union == 0 ? 0.0 : (double) common.size() / union;

        int ageDiff = Math.abs(viewer.age(context.today()) -
candidate.age(context.today()));
        double ageScore = Math.max(0.0, 1.0 - (double) ageDiff / AGE_SPAN_YEARS);

        boolean sameCity = viewer.isInSameCityAs(candidate);
        double cityScore = sameCity ? 1.0 : 0.0;

        double total = WEIGHT_INTERESTS * interestScore + WEIGHT_AGE * ageScore +
WEIGHT_CITY * cityScore;

        List<String> reasons = new ArrayList<>();
        if (!common.isEmpty()) {
            reasons.add("Общие интересы: " + describe(common));
        }
        if (sameCity) {
            reasons.add("Тот же город");
        }
        if (ageDiff <= CLOSE_AGE_YEARS) {
            reasons.add("Близкий возраст");
        }
        return new ScoredCandidate(candidate, clamp(total), List.copyOf(reasons));
    }

    private static String describe(List<Interest> common) {
        String listed =
common.stream().limit(MAX_LISTED_INTERESTS).map(Interest::getName).collect(Collectors.
joining(", "));
        int rest = common.size() - MAX_LISTED_INTERESTS;
        return rest > 0 ? listed + " и ещё " + rest : listed;
    }
}

```

Листинг Г.3 – Гибридная стратегия HybridStrategy

```

package com.matchly.recommendation.strategy;

import com.matchly.profile.Profile;
import com.matchly.recommendation.RecommendationContext;
import com.matchly.recommendation.ScoredCandidate;
import com.matchly.recommendation.StrategyType;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

/**
 * Гибрид (паттерн «Компоновщик» поверх стратегий): взвешенная сумма трёх базовых
оценок
 * плюс бонус за проявленный интерес - если кандидат уже лайкнул зрителя, шанс матча
выше.
 */
@Component
@RequiredArgsConstructor

```

```

public class HybridStrategy extends AbstractRecommendationStrategy {

    static final double WEIGHT_CONTENT = 0.5;
    static final double WEIGHT_COLLABORATIVE = 0.3;
    static final double WEIGHT_POPULARITY = 0.2;
    static final double LIKED_YOU_BONUS = 0.15;
    static final String LIKED_YOU_REASON = "Проявил(а) интерес к вам";

    private final ContentBasedStrategy contentBased;
    private final CollaborativeStrategy collaborative;
    private final PopularityStrategy popularity;

    @Override
    public StrategyType type() {
        return StrategyType.HYBRID;
    }

    @Override
    public List<ScoredCandidate> rank(Profile viewer, List<Profile> candidates,
RecommendationContext context) {
        Map<Long, ScoredCandidate> content = byProfile(contentBased.rank(viewer,
candidates, context));
        Map<Long, ScoredCandidate> collab = byProfile(collaborative.rank(viewer,
candidates, context));
        Map<Long, ScoredCandidate> popular = byProfile(popularity.rank(viewer,
candidates, context));

        return candidates.stream().map(candidate -> {
            Long id = candidate.getId();
            double score = WEIGHT_CONTENT * content.get(id).score()
                + WEIGHT_COLLABORATIVE * collab.get(id).score()
                + WEIGHT_POPULARITY * popular.get(id).score();
            LinkedHashSet<String> reasons = new LinkedHashSet<>();
            reasons.addAll(content.get(id).reasons());
            reasons.addAll(collab.get(id).reasons());
            reasons.addAll(popular.get(id).reasons());
            if (context.hasLiked(id, viewer.getId())) {
                score += LIKED_YOU_BONUS;
                reasons.add(LIKED_YOU_REASON);
            }
            return new ScoredCandidate(candidate, clamp(score), new
ArrayList<>(reasons));
        }).toList();
    }

    private static Map<Long, ScoredCandidate> byProfile(List<ScoredCandidate> scored)
{
        return scored.stream().collect(Collectors.toMap(s -> s.profile().getId(),
Function.identity()));
    }
}

```

Листинг Г.4 – Конфигурация безопасности SecurityConfig

```

package com.matchly.security;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;

```

```

import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.web.SecurityFilterChain;

/**
 * Конфигурация Spring Security: stateless-режим с JWT.
 * <ul>
 * <li>публичные: регистрация, вход, Swagger, health, статика интерфейса;</li>
 * <li>{@code /api/admin/**} - только роль ADMIN;</li>
 * <li>остальной {@code /api/**} - любой аутентифицированный пользователь.</li>
 * </ul>
 */
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtDecoder jwtDecoder;
    private final JwtUserAuthenticationConverter authenticationConverter;
    private final ProblemDetailAuthenticationEntryPoint authenticationEntryPoint;
    private final ProblemDetailAccessDeniedHandler accessDeniedHandler;

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/api/auth/register",
                    "/api/auth/login").permitAll()
                .requestMatchers("/v3/api-docs/**", "/swagger-ui/**",
                    "/swagger-ui.html").permitAll()
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/**").authenticated()
                .anyRequest().permitAll())
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt ->
                    jwt.decoder(jwtDecoder).jwtAuthenticationConverter(authenticationConverter))
                .authenticationEntryPoint(authenticationEntryPoint)
                .accessDeniedHandler(accessDeniedHandler))
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authenticationEntryPoint)
                .accessDeniedHandler(accessDeniedHandler));
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }
}

```

Листинг Г.5 – Глобальный обработчик ошибок GlobalExceptionHandler

```

package com.matchly.common.api;

import com.matchly.common.exception.MatchlyException;
import jakarta.servlet.http.HttpServletRequest;
import lombok.extern.slf4j.Slf4j;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCodes;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;

```

```

import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.AccountStatusException;
import org.springframework.security.authentication.BadCredentialsException;
import
org.springframework.security.authentication.InsufficientAuthenticationException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.oauth2.core.OAuth2AuthenticationException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.Objects;

/**
 * Единая точка обработки ошибок. Любое исключение превращается в ответ формата
 * RFC 9457 (application/problem+json) и записывается в лог с нужным уровнем.
 * Наследование от {@link ResponseEntityExceptionHandler} даёт готовую обработку
 * стандартных ошибок Spring MVC (неверный JSON, неподдерживаемый метод и т.п.).
 */
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    /** Прикладные исключения: статус и заголовок берутся из самого исключения. */
    @ExceptionHandler(MatchlyException.class)
    public ProblemDetail handleMatchly(MatchlyException ex, HttpServletRequest
request) {
        log.warn("{} {} -> {} {}", request.getMethod(), request.getRequestURI(),
            ex.getStatus().value(), ex.getMessage());
        return problem(ex.getStatus(), ex.getTitle(), ex.getMessage());
    }

    /** Нет или недействителен токен: сюда же делегируют entry point и access denied
handler безопасности. */
    @ExceptionHandler(AuthenticationException.class)
    public ProblemDetail handleAuthentication(AuthenticationException ex,
HttpServletRequest request) {
        String detail = switch (ex) {
            case AccountStatusException e -> e.getMessage();
            case BadCredentialsException e -> e.getMessage();
            case InsufficientAuthenticationException e -> "Authentication is
required";
            case OAuth2AuthenticationException e -> "Invalid or expired token";
            default -> "Authentication failed";
        };
        log.debug("{} {} -> 401 {}", request.getMethod(), request.getRequestURI(),
ex.getMessage());
        return problem(HttpStatus.UNAUTHORIZED, "Unauthorized", detail);
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ProblemDetail handleAccessDenied(AccessDeniedException ex,
HttpServletRequest request) {
        log.warn("{} {} -> 403 access denied", request.getMethod(),
request.getRequestURI());
        return problem(HttpStatus.FORBIDDEN, "Forbidden", "Access is denied");
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ProblemDetail handleIntegrity(DataIntegrityViolationException ex,
HttpServletRequest request) {
        log.warn("{} {} -> 409 data integrity violation: {}", request.getMethod(),
request.getRequestURI(),
            ex.getMostSpecificCause().getMessage());
    }
}

```

```

        return problem(HttpStatus.CONFLICT, "Conflict", "Request conflicts with
existing data");
    }

    /** Последняя линия обороны: неожиданная ошибка логируется со стеком, клиенту
уходит нейтральный ответ. */
    @ExceptionHandler(Exception.class)
    public ProblemDetail handleUnexpected(Exception ex, HttpServletRequest request) {
        log.error("{} {} -> 500 unexpected error", request.getMethod(),
request.getRequestURI(), ex);
        return problem(HttpStatus.INTERNAL_SERVER_ERROR, "Internal Server Error",
"Unexpected error occurred");
    }

    /** Ошибки валидации тела запроса: добавляем список полей с сообщениями. */
    @Override
    protected ResponseEntity<Object>
handleMethodArgumentNotValid(MethodArgumentNotValidException ex, HttpHeaders headers,
HttpStatus code,
status, WebRequest request) {
        List<Map<String, String>> errors =
ex.getBindingResult().getFieldErrors().stream()
            .map(fe -> Map.of("field", fe.getField(),
"message", Objects.requireNonNullElse(fe.getDefaultMessage(),
"invalid value")))
            .toList();
        ProblemDetail pd = problem(HttpStatus.BAD_REQUEST, "Validation failed",
"Request contains invalid fields");
        pd.setProperty("errors", errors);
        log.debug("Validation failed: {}", errors);
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(pd);
    }

    private static ProblemDetail problem(HttpStatus status, String title, String
detail) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(status, detail);
        pd.setTitle(title);
        pd.setProperty("timestamp", Instant.now());
        return pd;
    }
}

```

ПРИЛОЖЕНИЕ Д. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Таблица Д.1 – Автотесты JUnit 5 (все успешно пройдены)

Тестовый класс	Вид	Что проверяется	Тестов
AuthServiceImplTest	unit	регистрация, вход, ошибки учётных данных, блокировка	6
JwtTokenServiceTest	unit	выдача и проверка JWT, отклонение чужого ключа	2
ProfileTest	unit	возраст, инварианты анкеты, совместимость предпочтений, общие интересы	7
MatchTest	unit	упорядочивание пары, собеседник, запрет матча с самим собой	3
ImageTypeDetectorTest	unit	JPEG, PNG, WebP, неизвестные и короткие данные	3
ContentBasedStrategyTest	unit	веса составляющих, максимум и ноль, сокращение причин	6
CollaborativeStrategyTest	unit	косинусная близость, взвешивание, холодный старт	4
PopularityStrategyTest	unit	нормализация по максимуму, формы множественного числа	3
HybridStrategyTest	unit	взвешенная сумма, бонус за встречный лайк, ограничение единиц	2
AuthApiTest	API	регистрация, вход, ошибки, удаление аккаунта, публичные адреса, 404/405/415	15
AdminUsersApiTest	API	права, поиск, блокировка, удаление, лимиты пагинации	6
AdminStatsApiTest	API	статистика администратора	1
ProfileApiTest	API	анкеты, валидация, карточки, фото, границы, каскады, блокировка владельца	15
InterestApiTest	API	справочник, CRUD администратора, каскадное удаление из анкет	4
ReactionApiTest	API	лайк, пропуск, матч, разрыв, ошибки	6
RecommendationApiTest	API	фильтрация, порядок по стратегиям, параметры, настройка по умолчанию	4
DemoDataSeederTest	API	генерация демо-данных и её идиоматичность	1
MatchlyApplicationTests	API	запуск контекста и применение миграций	1
Итого			89

Таблица Д.2 – Сквозные браузерные сценарии Playwright

Сценарий	Содержание	Результат
e2e.js, 18 шагов	регистрация, онбординг с фото, лента, взаимный лайк и матч, матчи, пропуск клавиатурой, редактирование анкеты, разрыв матча, панель администратора (статистика, пользователи, интересы, настройки), мобильная ширина	пройден, ошибок консоли и ответов 5xx нет
corner.js, 19 групп	маршруты без сессии, испорченный токен, переводы ошибок, глубокие ссылки, валидация, лимит интересов, провал фото, пустая лента, XSS, сохранение алгоритма, двойной клик, горячие клавиши, блокировка во время работы, удаление аккаунта, поиск и пагинация, мобильная верстка всех экранов	пройден локально и против стека Docker

